

SC2002 - OBJECT ORIENTATED DESIGN & PROGRAMMING

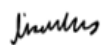
Project Title: InPlace Management System

Declaration of Original Work for SC2002 Assignment

We hereby declare that the attached group assignment has been researched, undertaken, completed, and submitted as a collective effort by the group members listed below.

We have honored the principles of academic integrity and have upheld Student Code of Academic Conduct in the completion of this work.

We understand that if plagiarism is found in the assignment, then lower marks or no marks will be awarded for the assessed work. In addition, disciplinary actions may be taken.

Name	Course	Lab Group	Signature / Date
Chu Heng Yi Terence	SC2002	SCE2 Group 2	 21/11/2025
Tan Chun Ee, Jovan	SC2002	SCE2 Group 2	 21/11/2025
Wee Kiat Ming, Nicholas	SC2002	SCE2 Group 2	 21/11/2025
Wong Xueqi	SC2002	SCE2 Group 2	 21/11/2025
Yong Zi Hong	SC2002	SCE2 Group 2	 21/11/2025

Assumptions Made

In our interpretation of the Assignment, we assume that the statement “Able to create internship opportunities (up to 5) for their companies” under the Company Representatives’ user capabilities requirements does not mean that only 5 opportunities can be created for a company, but rather that 5 opportunities can be created per company representative, even if they work for the same company. This is because if company representatives are from different departments of a big company, they should be able to offer several different opportunities.

We also assume that the system is run on a command line interface with only one user accessing the application at a time.

Design Considerations

Our main design consideration was to have our project adhere to the SOLID design principles to keep our design as extensible, maintainable and reusable as possible.

The OO concept of encapsulation is clearly used throughout our code as only the necessary methods used by other classes are set as public while all other methods are private. All attributes of classes are private as well, with only the necessary setters and getter methods implemented with the appropriate access modifiers.

We adopted the Entity-Control-Boundary (ECB) pattern in planning out our project design, mainly (and initially unknowingly) sticking to the Single Responsibility Principle (SRP). We had a control class to manage each entity class (e.g. Student entity class has StudentManager control class), such that each control class would only be responsible for updating its own corresponding entity class. This was our initial attempt to limit the reasons for our classes to change, which mostly succeeded in adhering to the SRP for simpler classes, except for certain Control classes.

However, due to our design strictly following ECB and the late introduction of the SOLID design principles to our design, especially the Interface Segregation Principle (ISP) and Dependency Inversion Principle (DIP), we ended up with many interdependent classes, leading to a tightly coupled design. As a result, we faced difficulties using abstractions in our design. Hence the code does not fully demonstrate our understanding of the DIP, but we managed to demonstrate the SOLID design principles through these parts of our code:

SOLID Design Principles

1. SRP: Specialised Classes

Our design is such that each control class manages only 1 entity class, while boundary classes need only focus on user input and call on control classes to handle the logic. As such, the entity classes have no reason to change unless their individual attributes change, while control classes have no reason to change unless specific logic changes, and boundary classes only change if the type of user input expected changes.

Some examples include:

Entity – Student (changes if attributes change)

Control – FilterManager (changes if any filter options are introduced or removed;
ReportManager (changes if report format changes)

Boundary – CareerStaffMainPage (changes if UI change)

2. Open-Closed Principle (OCP) & Liskov Substitution Principle (LSP): User relationship with FilterManager

Our implementation of User as an abstract class with subclasses Student, CareerStaff and CompanyRepresentative inheriting from it does not only demonstrate the OO concepts of abstraction and inheritance, but also adheres to OCP. This is because the abstract User base class is open to extension without requiring modifications to the existing code.

For example, a new type of User can be implemented by inheriting from the User class without affecting FilterManager, because FilterManager only depends on the getFilterSettings() method which all User subclasses inherit from the User base class. This shows that the FilterManager class is closed to modifications while the User class is open to extension.

Additionally, FilterManager also applies LSP through its updateFilterSettings method, which calls getFilterSettings(). As explained above, all User subclasses inherit that method, so polymorphism ensures that the correct getFilterSettings() behaviour is applied for each subclass. This allows FilterManager's updateFilterSettings(...) method to function regardless of the specific concrete subclass (e.g. Student, CareerStaff, CompanyRepresentative) taken in by the method. Since the updateFilterSettings(...) method accepts a User object, any subclass

of the User class can be substituted for the base class without affecting the correctness of FilterManager's behaviour.

Therefore, FilterManager and its relationship with the User class demonstrate adherence to OCP and LSP, while applying the OO concepts of abstraction, inheritance and polymorphism.

```
1 package control;
2
3 import entity.*;
4 import entity.Enums.InternshipLevel;
5 import entity.Enums.OpportunityStatus;
6
7 public class FilterManager {
8     public boolean updateFilterSettings(User user, boolean yearOfStudySet, boolean majorSet,
9         boolean internshipOpportunityStatusSet, boolean internshipLevelSet,
10         boolean dateSet, boolean visibilitySet, boolean companyNameSet,
11         int yearOfStudy, String major, OpportunityStatus opportunityStatus,
12         InternshipLevel internshipLevel, String startDate, String closingDate,
13         Boolean visibility, String companyName) {
14         FilterSettings filterSettings = user.getFilterSettings();
15         if(filterSettings == null){
16             System.out.println(x: "Filter Settings not found.");
17             return false;
18         }
19         filterSettings.setYearOfStudySet(yearOfStudySet);
20         filterSettings.setMajorSet(majorSet);
21         filterSettings.setInternshipOpportunityStatusSet(internshipOpportunityStatusSet);
22         filterSettings.setInternshipLevelSet(internshipLevelSet);
23         filterSettings.setDateSet(dateSet);
24         filterSettings.setVisibilitySet(visibilitySet);
25         filterSettings.setCompanyNameSet(companyNameSet);
26         filterSettings.setYearOfStudy(yearOfStudy);
27         filterSettings.setMajor(major);
28         filterSettings.setInternshipOpportunityStatus(opportunityStatus);
29         filterSettings.setInternshipLevel(internshipLevel);
30         filterSettings.setStartDate(startDate);
31         filterSettings.setCloseDate(closingDate);
32         filterSettings.setVisibility(visibility);
33         filterSettings.setCompanyName(companyName);
34         return true;
35     }
36 }
37
```

3. ISP: IPassword and IFilter Interfaces

Adhering to ISP, we chose to separate the IPassword and IFilter Interfaces instead of combining them into a single, large interface. This allows classes such as CompanyRepMainPage and StudentMainPage to retain the same functionality by implementing both interfaces without forcing other classes to implement unnecessary methods. For example, the CareerStaffMainPage only requires the IPassword interface and can implement the interface independently without having to implement unrelated methods from IFilter.

4. DIP: UserMainPage Interface

Following DIP, LoginManager depends on the UserMainPage interface (abstraction) that each user's HomePage implements. Each User subclass provides its concrete page to

LoginManager through this interface, which LoginManager then uses to direct the LoginPage. This decouples high-level logic from low-level UI classes, satisfying DIP.

The 4 examples above demonstrate our use of SOLID design principles and OO concepts, which helped ensure reusability, extensibility and maintainability in our design.

Enumerators

Apart from SOLID, our use of enumerators also helps to enforce extensibility and easy maintenance, as it does not only keep the code readable, but also allows us to introduce new statuses to implement new functionalities with minor changes. For example, if we update our system to store withdrawn applications so students can refer to their past applications, we can simply add a 'WITHDRAWN' under our ApplicationStatus Enums and implement the corresponding logic for this scenario, without having to edit any of our other code.

Design Patterns

Our team has made several deliberate trade-offs in our design. Applying DIP and ISP increased the number of classes and interfaces, but this was justified by the gain in modularity, testability, and ease of extension. Centralising all data management in a single Database class slightly relaxes SRP but simplifies the architecture for the scale of this project. Additionally, Manager classes introduced extra layers, but they serve as Facades to simplify the boundary classes' interaction with the system.

Since we learned the design patterns later in the project, we considered but did not adopt patterns such as Observer, Decorator, Adapter, and Singleton, as they required additional infrastructure or introduced unnecessary complexity. Nonetheless, our design naturally aligns with the following patterns:

1. Factory pattern

LoginManager depends on the UserMainPage interface and receives the appropriate HomePage object from each User subclass instead of creating it directly. This mirrors the Factory Pattern as object creation is delegated to subclasses through an abstraction, keeping LoginManager decoupled from the boundary classes' implementation details.

2. Strategy pattern

Our filtering architecture resembles the Strategy Pattern, where each User subclass provides its filter preferences through `getFilterSettings()`, and `FilterManager` applies the correct behaviour at runtime using polymorphism. This keeps the filtering logic flexible and easily extendable without modifying `FilterManager`.

3. Facade Pattern

Manager classes such as `OpportunityManager`, `ApplicationManager`, and `FilterManager` function like Facades. They expose simplified, high-level methods for other classes to call, while encapsulating complex logic involving validation, multiple entity types, and data retrieval from the Database. Although not intentionally designed as Facades, their behaviour aligns closely with the pattern's goal of hiding subsystem complexity.

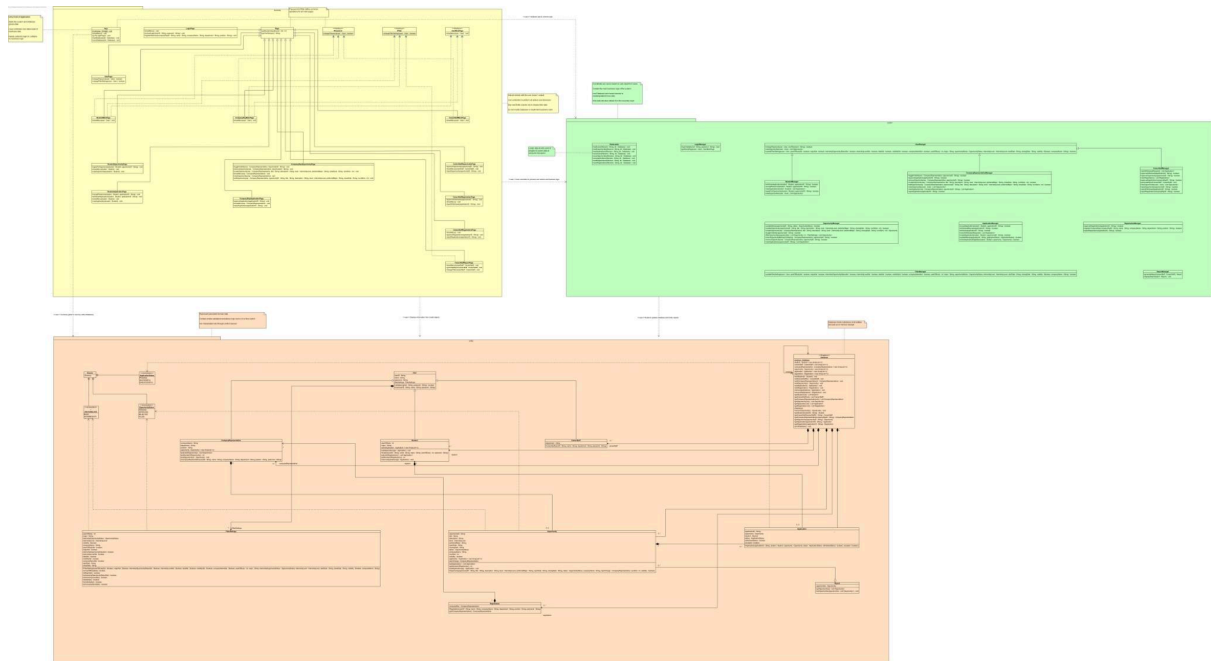
4. Repository Pattern

Our Database class acts as a Repository by providing centralised, collection-based access to domain objects. Instead of allowing direct manipulation of lists of `Student`, `Opportunity`, etc, Database encapsulates storage operations like `add`, `remove`, and `get`. This keeps persistence logic isolated, ensures consistent access patterns across the system, and reduces coupling between higher-level classes and data structures.

Overall, the 4 patterns we used balance keeps our design simple, extensible and maintainable.

UML Class Diagram

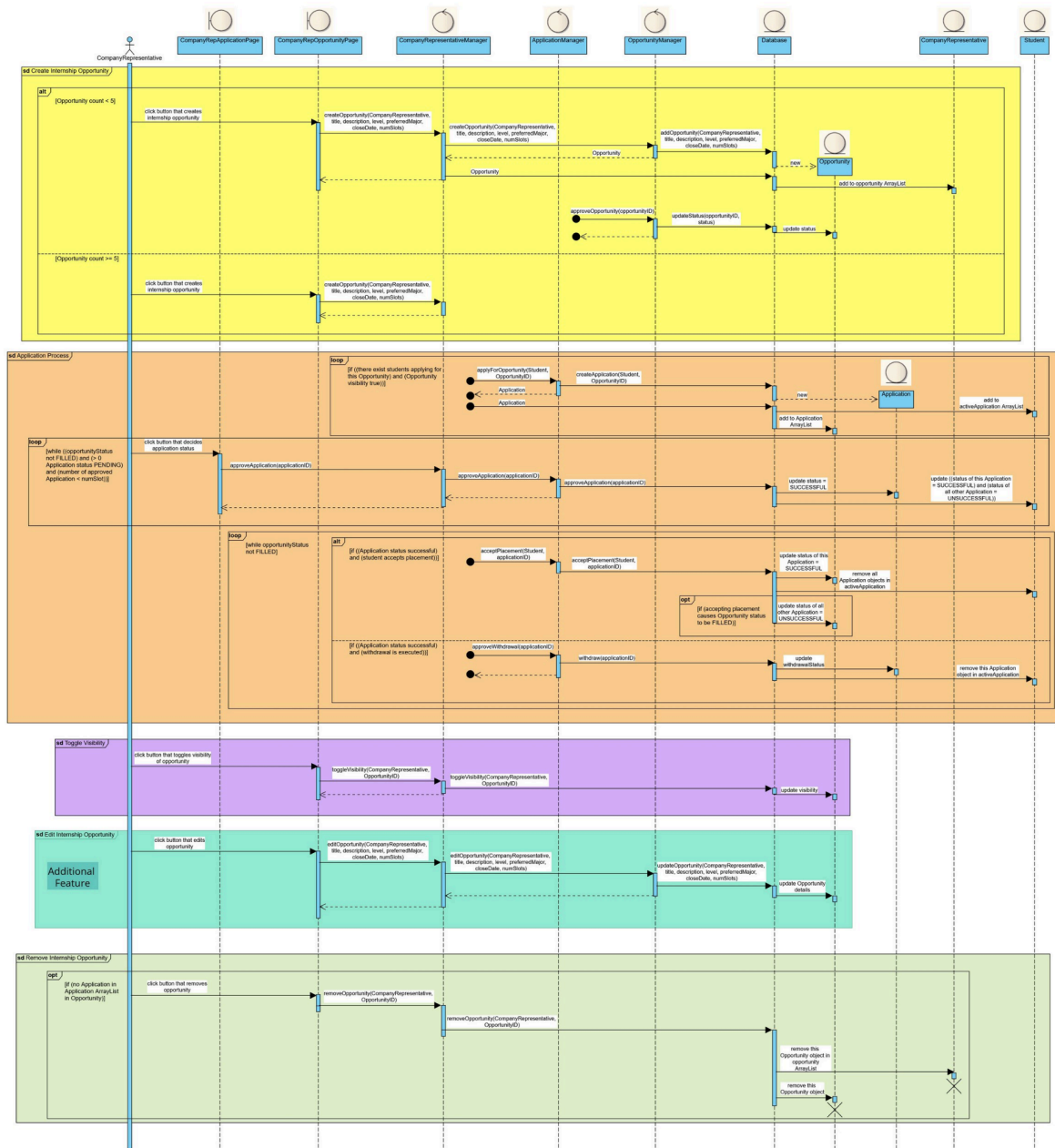
The following image shows a UML Class Diagram with notations that illustrate the dependencies between the different classes, with notes to explain the internal collaborations inside the Controllers' box, since our high-level class diagram intentionally does not have the dependencies between control classes drawn due to the presence of too many interconnections.



(see GitHub [uml class diagram.jpg](#) for clearer image)

UML Sequence Diagram

The following image depicts a UML Sequence Diagram with notations that illustrate the process of a company representative's role in internship management as well as in internship application.



(see GitHub [sequence diagram.jpg](#) for clearer image)

Testing

Please refer to GitHub [Test Cases and Output.pdf](#) document.

Reflection

In the process of completing this project, our team faced challenges designing the application.

We initially designed the system using the ECB pattern before learning the SOLID principles. Although ECB helped us maintain SRP for most classes, and our abstract User class provided some level of OCP, we later discovered some violations of S and O of SOLID, along with several tightly coupled control classes.

After learning SOLID, we had trouble implementing abstractions to reduce the dependencies between the classes and were unable to fully refactor the existing code to follow SOLID given our limited time. Hence, we focused our fixes on specific segments of our code to demonstrate our understanding of SOLID in the design considerations section above.

However, we recognise that our design would greatly benefit more abstractions through the use of several interfaces. The interfaces would allow us to extend the SOLID design principles across several classes, decoupling many of the control classes and greatly improving the extensibility of our application.

Currently, OpportunityManager and ApplicationManager expose their methods to all user types despite each user only using a subset of all the methods. This leads to heavy reliance on the two concrete classes, resulting in unnecessary coupling.

To decouple the dependencies, we could introduce 6 role-specific (student, company representative, career staff) interfaces:

- 3 for Opportunity services (e.g. IStudentOpportunityService)
- 3 for Application services (e.g. ICareerStaffApplicationService)

OpportunityManager and ApplicationManager would implement their 3 relevant interfaces, while each user manager class (e.g. StudentManager) would implement their 2 respective services. This design adheres to SOLID because:

1. SRP - OpportunityManager and ApplicationManager only have 1 reason to change

The individual interfaces will now manage all changes, instead of the huge OpportunityManager class, and now OpportunityManager changes only if the logic for handling opportunities changes. Similarly, ApplicationManager changes only if the logic for handling applications changes. Hence, the 2 classes now adhere to SRP.

2. OCP - User Managers depends only on interfaces, not concrete classes

The interfaces allow the system to be open for extensions such as adding of new roles or behaviours (like dashboards, reports not only for career staff, etc) while keeping the system

closed to modification, meaning that existing entity, control and boundary classes do not have to change (except if completely new logic has to be implemented, which is reasonable for OCP). As such, only new classes or interfaces have to be added if the system were to have any extensions, thus it would satisfy OCP.

3. LSP - Use of abstraction in the system

With the appropriate use of abstractions in the system, all user types inherit from the User base class and implement the relevant abstract methods from the User class. This consistent use of abstraction across the design allows subclasses to be substituted for the User base class through polymorphism without breaking functionality and follows LSP.

4. ISP - Separation into specific interfaces

Separating the huge OpportunityManager and ApplicationManager classes into multiple small, role-specific interfaces ensures each class only depends on the methods they need, preventing unnecessary method exposure.

5. DIP - Dependencies on concrete classes removed

Control classes now depend on interfaces (abstraction) instead of concrete OpportunityManager and ApplicationManager classes through the use of polymorphism. The interfaces do not rely on the internal details of either manager classes, and the concrete managers depend on the abstractions they implement.

For example, StudentManager now calls methods from OpportunityManager through the interface and does not actually have to know any details from OpportunityManager. Since the interfaces make no reference to the internal data structures or logic inside OpportunityManager, the interface will not change even if the implementation changes.

Therefore, the abstractions help eliminate the tight coupling in our design.

As such, the implementation of the aforementioned abstractions employ the concept of polymorphism and would greatly reduce the dependencies in our system, making our application more extensible, easier to maintain, and making our code more reusable.

This has emphasised the importance of abstraction to our team, and we have learned to introduce SOLID principles into our design earlier to prevent such a tightly coupled system.

Future Extensions

Besides design related changes, we believe a search function could be implemented in the future using Stream API, which can allow users to find opportunities or applications by name or description. For example, if students remember the details of certain opportunities they see or if company representatives remember details of applications they wish to review, the users would be able to search for them easily.

[GitHub Repository](#) + [Javadoc](#)